AgriAgent Delivery Schedule

Who does what, every week, from 12 October to the November submission

AgriAgent Final Year Project · 12 October 2026

Read this first. This is the team's schedule from today until the final submission and viva in the **second week of November (9–13 Nov)**. It tells every member exactly what to do each week and how we know it is done. Find your name in Section 6 (week by week) and on your own page in Section 9. The plan and the deliverables themselves are defined in `docs/AgriAgent_Project_Map.pdf`; this document is the **how and when**. If the two ever disagree, raise it in the group. Do not silently pick one.

Update 17 October (overrides this document where they differ). Decided on 17 October; details and evidence in `docs/AgriAgent_Week3_Dossier.pdf`. **Final submission and viva: Tuesday 10 November** (code freeze stays Thursday 5 November). **Models:** only Grounding DINO-tiny and SAM 2-tiny run locally; reasoning uses Groq and Cloudflare. Local Qwen and local Clef are dropped, and so is Kaggle compute (Kaggle only hosts downloads). **Evaluation set:** `eval_manifest_v1` failed the audit (fabricated labels); `eval_manifest_v2` (30 sugar beet + 8 cotton, official labels) is frozen. **Testing:** Sahil = validation, Sumant = system; Amit moves to development and the demo. **Hard deadline for the team hand-off tasks (dossier Section 5): Wednesday 21 October.** Known issues in Guide 2 §10 are now called K1–K12.

1 Key dates

Date	What happens
Mon 12 – Wed 14 Oct	Repo frozen. Everyone studies the Project Map and the study guides (`docs/guides/`).
Thu 15 Oct	Work starts. First real run of the vision models on the team laptop.
Sat 17 Oct	**Checkpoint 1:** first real local run posted; install smoke test done; test plans drafted.
Wed 21 Oct	Evaluation image set frozen (no more changes to which images we measure on).
Sun 25 Oct	**Checkpoint 2:** baseline real metrics table, including crop pixels on the spray map.
Thu 29 Oct	Weed-vs-crop improvement handed to evaluation.
Sun 1 Nov	**Checkpoint 3:** before/after numbers; test cycle 1 finished; every defect logged.
Wed 4 Nov	Regression test cycle 2 (re-test all fixes).
Thu 5 Nov	**Code freeze**, release tag `v1.0.0`, demo video recorded, full report draft.
Fri 6 – Sun 8 Nov	Diwali weekend (check dates). Light load: personal viva preparation only.
Mon 9 – Tue 10 Nov	Final checks; **submission and viva on Tue 10 Nov** (updated 17 Oct).

Why the freeze is Thursday 5 November, not the 6th. The Project Map said 6 November. Diwali falls around 6–8 November this year (check the exact dates), so we freeze one day earlier and do not plan any coding work over the festival.

2 Where we stand today (verified 12 October)

The question was: **is Week 2 done?** Week 2 had two different definitions, so both are checked. Everything marked "checked" below was run or read today, not copied from older documents.

2.1 Against our own Week 2 plan (Project Map and Week 2 dossier): done

Week 2 item	Status	Evidence (12 Oct)
Repository cleaned, contradicting docs removed	done	Repo has README, Project Map, Week 1/2 dossiers and five study guides only.
Spray-map export safety bug fixed	done	`tests/test_spray_export_safety.py` passes.
CI green; tests pass	done	Checked: **175 unit tests and 35 end-to-end tests pass**, `ruff check` clean, in a fresh environment set up like CI.
Model choices frozen	done	Project Map, "Model decisions (frozen)".
Honest look at the one real result	done	Week 2 dossier, Section 4.

2.2 Against the original proposal's Week 2 milestone: not done, carried into Week 3

The August proposal's Week 2 milestone was *"a command-line pipeline running zero-shot prompt to mask generation"*. That is only partly true:

Item	Status	What is missing
Real pipeline script (`scripts/run_live_pipeline.py`)	PARTIAL	Ran once on Kaggle (3 Oct). **Never run on our own laptop.** The laptop has no project environment yet (no `torch` installed).
Script default prompt	PARTIAL	Default `"cotton weed . broadleaf plant ."` is rejected by the safety gate, so it never sprays (known issue 4).
Deterministic safety router (System 1)	REAL	Code + tests. Two design bugs remain (known issues 1 and 2).
LangGraph assembly	PARTIAL	`create_langgraph_workflow()` exists; the plain-Python runner is what actually runs. Fine as is.
Agronomy knowledge base ingested	PARTIAL	Works offline in tests. Never used with a real model.
Evaluation set (`data/manifests/eval_manifest_v1.json`)	PARTIAL	20 images (10 sugar beet, 10 cotton) with polygons. **Where these annotations came from is not recorded** (added by an AI-written commit on 11 Oct). Some cotton "weed" polygons are thin slivers a few pixels tall. They must be checked against the original dataset before any number is reported.

> **Conclusion.** Week 2 achieved what **we** planned (cleanup, safety, honesty), but the project is about **one week behind** the original proposal: we still have no real run on our own hardware and no trusted ground truth. Week 3 therefore starts with exactly those two things. Everything else depends on them.

3 What “delivered” means in November

3.1 Base papers and main objectives

The project is built on two foundation-model papers, used together as in the open-source Grounded-SAM-2 pipeline:

- **Grounding DINO** (Liu et al., ECCV 2024): open-vocabulary detection from a text prompt.
- **SAM 2** (Ravi et al., 2024): promptable segmentation from boxes and points.

Supporting papers: **MedSAM-Agent** (2026) inspired the refinement loop; **Chong et al.** (IROS 2025, zero-shot crop/weed segmentation) is the closest related work and our comparison point in the report.

#	Main objective (from the proposal, scaled to what we can prove)	Today	Done by
O1	Zero-shot weed detection + segmentation from a text prompt with Grounding DINO + SAM 2, no training on our data, **running on our own laptop**.	REAL once (Kaggle)	Sat 17 Oct
O2	Refinement loop: when SAM 2’s predicted quality is low, add a corrective point and re-segment.	PARTIAL	Sun 1 Nov (measured on real images)
O3	Deterministic safety gate: only approved weed masks reach the spray map / GeoJSON; crop never sprayed; known gate bugs fixed.	PARTIAL	Sun 1 Nov
O4	Agronomy reasoning: when unsure, escalate to a reasoning model; herbicide advice only from the ICAR bulletin.	MOCK	Sun 1 Nov (one real Groq run)
O5	Real evaluation on annotated images: IoU/Dice, weed recall, **crop pixels on the spray map**, herbicide savings; plus one improvement with before/after numbers.	MOCK	Sun 1 Nov
O6	Working demo: Streamlit app on real photos + GeoJSON + recorded video.	MOCK	Thu 5 Nov

3.2 Measurable finish line

We are finished when each of these is true. They are the acceptance criteria for the final test.

1. The app runs in **real mode** on the team laptop on 5 prepared field photos, and the screen clearly says REAL or MOCK.
2. A frozen evaluation set of **at least 20 images** (target 40) with checked crop **and** weed annotations.
3. A results table: weed IoU / Dice, weed pixel recall, **percentage of crop pixels on the spray map**, herbicide savings, average seconds per image, each labelled with model, device and “real run”.
4. The same table **before and after** one weed-vs-crop improvement. If the improvement does not help, we report that honestly with the reason. A measured negative result is still a result; an unmeasured claim is not.
5. Known issues 1, 2, 4 and 5 fixed, each with a test; all tests and CI green on `main` at the freeze.
6. One real end-to-end Groq call shown, and 5 prescriptions checked by hand against the ICAR bulletin.

7. Final report (Typst), slide deck, demo video in two places, and every member able to explain the whole pipeline and their own part.

3.3 Explicitly out of scope

These were in the August proposal but will **not** be built. Say so in the report as future work: the other four applications (disease, yield counting, etc.), real GPS / drone routes, a PDF prescription report, cloud deployment of the app, and any "under 40 ms" speed claim (we will report the speed we actually measure).

3.4 Models we use (updated 12 October)

Research on 12 October changed two earlier conclusions. **Clef Flash is no longer blocked:** Cloudflare hosts it with a free daily allowance, and llama.cpp can run it locally. **Groq's Qwen model accepts images,** so the "crop or weed?" check has three possible back-ends, and we will pick the best one by measurement. Every model only **advises**. The deterministic gate still decides what is sprayed.

Job	Primary	Backup	Today
Detect + segment	Grounding DINO-tiny + SAM 2-tiny on the laptop GPU	Same models on Kaggle T4	REAL once
Triage: "is this mask safe to spray, or escalate?"	**Clef Flash on Cloudflare Workers AI** (`@cf/cloudflare/clef-flash`): text + image, returns probabilities	Clef Flash GGUF on the laptop through `llama-server` (text only, 6.5 GB)	PLANNED
Crop-or-weed check of uncertain boxes	Compare three on the same image patches: Clef Flash (Cloudflare), Groq `qwen/qwen3.8-27b`, local `Qwen3.5-4B`. Keep the most accurate.	Deterministic rules only	PLANNED
Weed diagnosis + prescription text	Groq `qwen/qwen3.8-27b`	Local `Qwen3.5-4B` (llama-server), or Groq `openai/gpt-oss-120b` (text only)	PARTIAL
Herbicide choice and dose	**Not a model.** ICAR bulletin lookup + banned-list and dose checks in code.	-	PARTIAL
Safety authority	**Not a model.** `DeterministicRouter`	-	REAL (tests)

> **Two limits to remember. 8 GB GPU:** the vision models, a local Qwen server and a local Clef server cannot all be on the GPU at once. Run stages one after another, or use the hosted versions. **Groq free tier:** `qwen/qwen3.8-27b` allows about 30 requests/minute, 1,000/day, 8,000 tokens/minute and 200,000 tokens/day (12 Oct), and each image costs 2,048 tokens. That is about three image calls per minute, so bulk image checks go to Cloudflare or the local model, not Groq. The model is also marked **Preview** on Groq and could be withdrawn.

This updates the "frozen" model table in the Project Map. Chaitanya approved it on 12 October; the team confirms it at the Thursday 15 October call.

4 Roles

4.1 Development tracks (unchanged)

Member	Builds	Owns in the repo
Chaitanya (Track 1)	Vision pipeline, safety gate fixes, weed-vs-crop improvement; **integration lead** (reviews and merges PRs).	`src/vision/`, `src/workflow.py`, `src/agents/`, `scripts/`
Sumant (Track 2)	All model back-ends: Groq, Cloudflare Clef Flash, local llama-server; prescription checks; crop-or-weed comparison; **report lead**. Helps teammates with key setup (Appendix A).	`src/agronomy/`, `config/prompts.yaml`, and `src/agents/clef_flash_router.py` (reviewed by Chaitanya)
Sahil (Track 3)	Evaluation set, metrics incl. crop protection, all real evaluation runs.	`data/`, `src/evaluation/`
Amit (Track 4)	Streamlit app in real mode, GeoJSON fixes, CI, demo video; **slides lead**.	`app/`, `.github/`, `tests/e2e/`

4.2 Testing roles (new)

Testing is done mostly by Sahil and Amit, so the people who wrote the code are not the only ones checking it. They test **different questions**, so they do not duplicate each other.

Member	Testing role	Tests
Sahil	**Validation tester**: "Is the output correct?"	Model outputs against ground truth; metric code against hand-computed examples; crop-on-map numbers; prescriptions against the ICAR bulletin (independently of Sumant).
Amit	**System tester**: "Does the system work for a user?"	Fresh install from the README; every app flow in mock and real mode; REAL/MOCK labelling; GeoJSON contents; error handling (bad image, no internet, no weights); weekly regression; demo acceptance checklist.
Chaitanya	**Test lead and final sign-off**	Reviews test plans, triages every defect (severity + owner), unit tests for Track 1, signs off the freeze. Chaitanya's Software Testing Lab02 test-case sheet and Lab03 IEEE

		829 test plan both use AgriAgent and are the starting point for the team's test cases.
Sumant	Developer testing only	Unit tests for his own code; fixes defects in Track 2.

> **Why the split is fair.** Sahil and Amit carry testing on top of their tracks. Sumant has no testing duty, so he owns every model back-end and leads the report; Amit, whose app is the demo, leads the slides. Chaitanya has the hardest technical work plus integration and defect triage. The weekly hour estimates in Section 5 keep every member at roughly 10–13 hours per week.

4.3 How work flows

- **Tasks and defects live in GitHub Issues.** One issue per task in this schedule (title starts with its ID, e.g. `T1.3`). Labels: `track-1`…`track-4`, `bug`, `sev-high` / `sev-med` / `sev-low`.
- **Every change is a pull request** from your track branch. Chaitanya reviews and merges; for Track 1 PRs, Amit or Sahil reviews. CI must be green.
- **Defect life cycle:** tester files an issue with steps, expected, actual, and a screenshot ⟶ Chaitanya sets severity and owner within a day ⟶ owner fixes in a PR that says `Fixes #n` ⟶ **the tester who filed it re-tests and closes it.** Developers never close their own defects.
- **Severity: high** = crop could be sprayed, mock output could be shown as real, or the demo flow crashes; **medium** = wrong number or misleading screen, with a workaround; **low** = cosmetic.
- **Weekly rhythm:** Monday 15-minute call (plan the week) · Saturday checkpoint post in the WhatsApp group: each member writes **done / not done / blocked**, with a link or screenshot as evidence · Sunday Chaitanya updates the weekly dossier.
- **AI-agent rules** from the Project Map still apply: one agent at a time, no "verified/complete" without pasted command output, no new models or dependencies without a team decision.

5 The plan at a glance

Estimated hours in brackets. Bold = the week's critical-path task (everything else waits on it).

Week	Chaitanya	Sumant	Sahil	Amit
W3 12–18 Oct	**Laptop setup + first real run**; issue 5 (8 h)	Team key setup help; real Groq call; 5-patch image check on 3 back-ends; report skeleton (12 h)	**Audit eval annotations**; download datasets; validation test plan (12 h)	Fresh-install smoke test; system test plan (11 h)
W4 19–25 Oct	Issues 4 + 1; prompt policy (DL viva week) (8 h)	Groq inside workflow; **Clef adapter (Cloudflare + local)**; prescriptions vs PDF (13 h)	**Freeze eval set; crop-on-map metric; baseline run** (13 h)	REAL/MOCK badge; issues 7, 8b, 9; app real mode (12 h)
W5 26 Oct–1 Nov	**Crop-exclusion improvement**; issue 2; speed measurement (13 h)	Crop-or-weed comparison of 3 back-ends; report ch. 1–2 (12 h)	**After-improvement run**; test cycle 1, validation (12 h)	Before/after view; **test cycle 1, system** (12 h)

W6 2–8 Nov	Fix defects; **freeze Thu**; report ch. 3, 4a, 7 (12 h)	Fix defects; ch. 4b; **assemble full report** (12 h)	Regression cycle 2; results chapter (11 h)	Regression cycle 2; **demo video**; **slide deck**; testing chapter (13 h)
W7 9–13 Nov	Mock viva lead; final sign-off	Final report; submission formalities	Results slides; viva prep	Final slides; demo laptop ready

6 Week by week

Each task has an ID (track number . task number) to use in issue titles. "Done when" is what you post as proof.

6.1 Week 3 · 12–18 Oct · Study, then first real run

Goal: the vision models run for real on our laptop, and we know whether our ground truth can be trusted.

Who	Task	Done when	Due
All	Study the Project Map + Guides 1–3 (repo frozen until Wed).	Can say the five sentences of Guide 0 without notes.	Wed 14
All	Create **your own** Groq key and Cloudflare account, fill your `.env`, run `python scripts/check_providers.py` (Appendix A). Ask Sumant if stuck.	Screenshot of the PASS lines posted (hide keys).	Sat 17
Chaitanya	**T1.1** Create the project environment on the laptop (Python 3.11 venv; PyTorch build with CUDA 12.8, which the RTX 5050 needs; download DINO-tiny + SAM 2-tiny weights).	`torch.cuda.is_available()` prints True; weights cached.	Thu 15
Chaitanya	**T1.2** First real local run on 3 sample images (`tests/fixtures/sample/`) with prompt `"weed . cotton plant ."`.	Output images + log with **no** "Falling back to mock_mode" line, posted to group. Note peak GPU memory from `nvidia-smi`.	Sat 17
Chaitanya	**T1.3** Fix issue 5: when real mode is requested and weights fail to load, stop with a clear error instead of silently using mock.	PR merged with a test.	Sun 18
Sumant	**T2.1** All three back-ends answer: Groq, Cloudflare Clef Flash, local `llama-server` with Qwen3.5-4B (Appendix A). One real call through `src/agronomy/providers/hosted_groq.py`.	`check_providers.py` all PASS on your machine; output in an issue comment.	Fri 16
Sumant	**T2.2** Image check by hand: cut 5 patches (3 weed, 2 cotton) from the sample images; ask each back-end "crop or weed?". Groq and Cloudflare need the image sent with the request; local Qwen needs its `mmproj` file.	5 × 3 answer table posted. Tells us whether T2.5 is worth doing.	Sun 18

Sumant	**T2.6** Get the department's report format; create the final report skeleton in Typst with chapter owners.	`docs/report/` skeleton committed.	Sun 18
Sahil	**T3.1** Audit `eval_manifest_v1.json`: for each of the 20 images, find the original dataset file and compare our polygons with the official annotation. Mark each image **trusted / fix / drop**.	Audit table posted; the provenance written into the manifest description.	Sat 17
Sahil	**T3.2** Download the evaluation datasets (Sugar Beets 2016 sample + the cotton dataset). Confirm which one has **both crop and weed** annotations: the crop-on-map metric needs crop pixels.	Datasets on disk; answer posted.	Sun 18
Sahil	**T3.3** Validation test plan: 10–15 test cases (input image, expected result, how measured).	Typst or sheet shared; Chaitanya reviewed.	Sun 18
Amit	**T4.1** Fresh-install smoke test on your own laptop: follow the README exactly, word for word. Log every step that fails or is unclear.	Issues filed for each failure; mock app runs.	Fri 16
Amit	**T4.2** System test plan: 20–25 test cases covering every screen, mock vs real, GeoJSON, errors. Start from Chaitanya's Lab02 sheet.	Shared; Chaitanya reviewed.	Sun 18

> **Week exit check.** Real run images posted (O1 done) · annotation audit done · both test plans reviewed · smoke-test issues filed. **If the laptop GPU setup fails by Sat 17:** run T1.2 on Kaggle (the notebook already works) and continue; do not lose the week on drivers.

6.2 Week 4 · 19–25 Oct · Baseline real numbers

Goal: the first honest metrics table on real images. Chaitanya has the Deep Learning viva this week, so Track 1 work is small and front-loaded.

Who	Task	Done when	Due
Chaitanya	**T1.4** Fix issue 4 and write the **prompt and label policy**: one default prompt that names both weed and crop (e.g. `"weed . cotton plant ."`) so crop boxes are detected and excluded. Same default in the script, the app and `config/prompts.yaml`.	PR merged; test shows weed sprayed, cotton not.	Mon 19
Chaitanya	**T1.5** Fix issue 1: pass each mask's real detection confidence and predicted IoU to the gate instead of `1.0`.	PR merged; test where a low-confidence weed is **not** sprayed.	Tue 20
Chaitanya	Run the vision models for Sahil's baseline if Kaggle is not used (laptop has the GPU).	Raw outputs handed to Sahil.	Fri 23

Sumant	**T2.3** Make System 2 call Groq for real inside the workflow (forced escalation on one image). In `hosted_groq.py`: send the image, set `reasoning_effort: "none"` and JSON mode.	Screenshot / log of a real prescription; status in Project Map moves to `REAL`.	Tue 20
Sumant	**T2.8** Rewrite `src/agents/clef_flash_router.py` as a small HTTP client with two backends: Cloudflare REST and local `llama-server /v1/systemone`, chosen by `CLEF_BACKEND`. Keep the rule: deterministic checks first, and **any error means escalate, never spray**.	PR reviewed by Chaitanya; tests with faked HTTP replies; one real Cloudflare answer logged.	Thu 22
Sumant	**T2.4** Fix issue 12: check the banned list and dose ceilings in `prescription.py` against the ICAR PDF; write a table of 5 cotton weeds with page numbers. Also issue 8a (hard-coded fallback weed `"Trianthema"`).	Table in the repo; PR merged.	Sat 24
Sahil	**T3.4** Freeze evaluation set v2 (target 40 images, minimum 20, all audited, crop + weed annotations).	`eval_manifest_v2.json` merged; nobody edits it after this.	**Wed 21**
Sahil	**T3.5** Add the **crop-on-spray-map** metric (crop pixels sprayed ÷ all crop pixels) and weed pixel recall to `src/evaluation/`; fix issue 10 (savings depend on OpenCV).	PR merged with tests on hand-computed examples.	Thu 22
Sahil	**T3.6** Baseline real run over the frozen set (Kaggle T4 or the laptop).	Results table with model, device, date, "real run".	**Sun 25**
Amit	**T4.3** Visible REAL / MOCK badge on every result screen (reads the actual mode used, not the checkbox).	PR merged; screenshot of both.	Wed 21
Amit	**T4.4** Fix issue 7 (confidence slider must change the spray map, or be removed), issue 8b (hard-coded herbicide in GeoJSON) and issue 9 (label GeoJSON coordinates as pixel-based, not GPS).	PRs merged with e2e tests.	Fri 23
Amit	**T4.5** Run the app in real mode (laptop GPU, or CPU on your laptop, which is slow but works).	Screen recording of one real image.	Sun 25
Amit, Sahil	Test cycle 0: smoke-test this week's merged fixes.	Defects filed.	Sun 25

> **Week exit check.** Baseline real metrics table exists (**the most important checkpoint of the project**) · issues 1 and 4 fixed · real Groq call shown · REAL/MOCK badge live.

6.3 Week 5 · 26 Oct – 1 Nov · Improvement and full test cycle 1

Goal: one measured weed-vs-crop improvement, and every feature tested once by the testers.

Who	Task	Done when	Due
Chaitanya	**T1.6** Crop-exclusion improvement: detect crop with its own prompt phrase, segment it, and **subtract crop masks from weed masks pixel by pixel** before the gate. Behind a switch so baseline and improved can both run.	PR merged; switch documented; handed to Sahil.	**Thu 29**
Chaitanya	**T1.7** Fix issue 2: run triage (now Clef Flash, from T2.8) **before** the spray map is approved; escalated masks are withheld until System 2 answers.	PR merged; test that an escalated mask is not sprayed.	Sat 31
Chaitanya	**T1.8** Measure real seconds per image (detect, segment, total) on the laptop; replace every "< 40 ms" comment with the measured number.	Numbers in Sahil's table.	Sun 1
Sumant	**T2.5** Crop-or-weed comparison: take 40 boxes from Sahil's frozen set (half crop, half weed, from ground truth), ask all three back-ends, measure accuracy and seconds per call. Recommend one for the pipeline.	Accuracy table (3 back-ends × 40 boxes) + recommendation; Chaitanya decides.	Thu 29
Sumant	**T2.7** Report chapters 1 (introduction, problem) and 2 (literature, base papers), first draft.	Draft in `docs/report/`.	Sun 1
Sahil	**T3.7** Run the improved pipeline on the same frozen set; before/after table; 3 example images where it helped and 3 where it failed.	Table + figures posted.	**Sun 1**
Sahil	**T3.8** Test cycle 1 (validation): run every validation test case; check Sumant's 5 prescriptions against the PDF yourself.	Results sheet; defects filed.	Sat 31
Amit	**T4.6** App shows before/after (baseline vs improved) for the same image, and the crop-on-map number. Sidebar shows which back-end answered triage and System 2 (Cloudflare / Groq / local / mock).	PR merged.	Thu 29
Amit	**T4.7** Test cycle 1 (system): execute all system test cases in mock and real mode.	Results sheet (pass/fail per case); defects filed.	**Sat 31**

Chaitanya	Triage all cycle 1 defects within a day: severity + owner.	Every defect has a label and owner.	Sun 1

> **Week exit check.** Before/after numbers exist · issues 1, 2, 4, 5 fixed · test cycle 1 complete · defect list triaged. **If the improvement does not reduce crop on the map:** keep the baseline as the demo default, report the negative result and the reason. Do not start a new approach after Sun 1.

6.4 Week 6 · 2–8 Nov · Fix, freeze, record

Goal: no high-severity defects, code frozen Thursday, demo recorded, full report draft.

Who	Task	Done when	Due
All devs	Fix every high and medium defect from cycle 1 in your own track. Low ones: fix only if trivial.	Each fix re-tested and **closed by the tester**.	Tue 3
Amit, Sahil	Test cycle 2 (regression): re-run all test cases on `main`.	Results sheet; zero open high-severity defects.	**Wed 4**
Chaitanya	**Code freeze**: final review, all checks green, tag `v1.0.0`. After this only report/slides change.	Tag pushed; CI link posted.	**Thu 5**
Amit	**T4.8** Record the demo video (script: Guide 4) on the frozen version, with Chaitanya. Copy to Drive and a USB stick.	Video link posted.	Thu 5
Chaitanya	Report ch. 3 (system design), 4a (vision + safety), 7 (conclusion, limits, future work).	Drafts in `docs/report/`.	Thu 5
Sahil	Report ch. 6 (results) + 4c (data and evaluation).	Draft.	Thu 5
Amit	Report ch. 5 (testing: plan, cycles, defect statistics) with Sahil, + 4d (UI).	Draft.	Thu 5
Sumant	Report ch. 4b (agronomy and model back-ends); **assemble the full report**.	Full draft PDF posted.	**Thu 5**
Amit	**Slide deck**: each member sends their 2–3 slides by Wed 4; Amit assembles one deck in one style.	Deck posted.	Thu 5
All	Fri 6 – Sun 8: festival. Optional: read the full report once and Guide 3.	-	-

6.5 Week 7 · 9–13 Nov · Submit and defend

Who	Task	Due
All	Read the full report; send corrections to Sumant.	Mon 9
Sumant	Final report PDF; print / submit as the department requires. Amit: final slides.	Tue 10
Chaitanya	Mock viva: each member presents their part and answers Guide 3 questions about **another** member's track.	Tue 10

Amit	Demo laptop ready: weights downloaded (offline), 5 good + 2 hard images, `.env`, video as backup. Run the Guide 4 checklist.	Day before viva
All	Submission and viva.	as scheduled

7 Testing plan (summary)

Level	What is checked	Who	When
Unit	Each function on fixed inputs (existing 175 tests + one per fix).	Developers	Every PR
Integration / e2e	Whole pipeline in mock mode (existing 35 scenarios).	Amit maintains	Every PR (CI)
System	The app as a user sees it, mock and real mode, install, errors.	Amit	Cycle 0, 1, 2
Validation	Accuracy against ground truth; metric correctness; prescriptions against the bulletin.	Sahil	Cycle 1, 2
Acceptance	The 7 finish-line criteria in Section 3.2 and the Guide 4 demo checklist.	All; Chaitanya signs	Thu 5 Nov

Entry rule for a test cycle: the code is merged to `main` and CI is green. **Exit rule for the freeze:** zero open high-severity defects, every medium defect either fixed or written into the report as a known limitation.

8 Risks and what we do if they happen

Risk	Plan
Only Chaitanya's laptop has a GPU (8 GB).	Sahil runs evaluation on Kaggle T4 (notebook already exists). Amit tests the app on CPU (slow but fine). Book laptop time in the Monday call.
PyTorch does not work with the new RTX 5050.	Needs a CUDA 12.8 PyTorch build. If not solved by Sat 17 Oct, use Kaggle for all real runs; the demo then uses the recorded video plus mock mode live.
Groq withdraws the Preview model `qwen/qwen3.8-27b`, or we hit the free limits.	Text reasoning switches to `openai/gpt-oss-120b` on Groq (change `GROQ_MODEL_NAME` only). Image checks use Cloudflare or local Qwen. Each member uses their **own** key, so limits are not shared.
Cloudflare asks for a card, or the free 10,000 Neurons/day run out.	Do not add a card. Use the local Clef Flash GGUF (text only) or the deterministic gate alone; triage is advisory, so the demo still works.
Two models on the GPU at once run out of memory.	Run one stage at a time; prefer hosted Clef and Groq during the demo; the local servers are the offline backup.
A key leaks (pasted in WhatsApp, committed to Git).	Revoke it in the console at once and create a new one. `.env` is in `.gitignore`; never put keys anywhere else.
Zero-shot models simply cannot separate weed from crop.	That **is** our research finding. Report the crop-on-map number honestly, show the crop-exclusion

	rule's effect, and propose fine-tuning as future work.
The annotations turn out to be wrong.	Drop those images; 20 checked images is the minimum. Never report a number on unchecked ground truth.
Exams, quizzes, festival, illness.	Critical-path tasks are bold; if you will miss one, say so in the group **before** the due date so someone else can take it.
An AI agent makes big unreviewed changes again.	No PR merges without a human who can explain it. Chaitanya rejects PRs that touch another track's files without that owner's review.

9 Your page

Cut-out summaries. Share each one with its owner. Each list is in order; do not skip ahead past a bold item. **Everyone, Week 3:** your own Groq key + Cloudflare account + `.env`, checked with `scripts/check_providers.py` (Appendix A), by Sat 17.

9.1 Chaitanya: Track 1 vision + safety, integration lead, test lead

- **W3:** laptop environment + weights (Thu 15) → **first real run posted (Sat 17)** → issue 5, fail loudly instead of silent mock (Sun 18). Review both test plans.
- **W4 (DL viva week, keep it small):** prompt/label policy + issue 4 (Mon 19) → issue 1, real confidence into gate (Tue 20) → GPU time for Sahil's baseline.
- **W5: crop-exclusion improvement (Thu 29)** → issue 2, triage before approval (Sat 31) → measure real speed (Sun 1) → triage all cycle 1 defects.
- **W6:** fix defects → **freeze + tag v1.0.0 (Thu 5)** → record video with Amit → report ch. 3, 4a, 7.
- **W7:** run the mock viva; final sign-off.
- **Every week:** review and merge PRs; update the weekly dossier on Sunday.

9.2 Sumant: Track 2 agronomy + model back-ends, report lead

- **W3:** your keys + all three back-ends answering, help the others with Appendix A (Fri 16) → 5-patch "crop or weed?" check on Groq, Cloudflare and local Qwen (Sun 18) → report skeleton in the department format (Sun 18).
- **W4:** real Groq call inside the workflow, with image + JSON mode (Tue 20) → **Clef Flash adapter: Cloudflare + local (Thu 22)** → prescriptions vs ICAR PDF for 5 weeds, issues 12 + 8a (Sat 24).
- **W5: crop-or-weed comparison: 3 back-ends × 40 boxes (Thu 29)** → report ch. 1–2 draft (Sun 1).
- **W6:** fix your defects → report ch. 4b → **assemble the full report (Thu 5)**.
- **W7:** final report; submission formalities.

9.3 Sahil: Track 3 data + evaluation, validation tester

- **W3: audit the 20 evaluation images against the original datasets (Sat 17)** → download datasets, confirm crop + weed annotations (Sun 18) → validation test plan (Sun 18).
- **W4: freeze evaluation set v2 (Wed 21)** → crop-on-map metric + weed recall + issue 10 (Thu 22) → **baseline real run and table (Sun 25)**.
- **W5: after-improvement run, before/after table + examples (Sun 1)** → test cycle 1 validation incl. independent prescription check (Sat 31).
- **W6:** regression cycle 2 (Wed 4) → report ch. 6 + 4c, testing chapter with Amit (Thu 5).
- **W7:** results slides; viva prep.

9.4 Amit: Track 4 UI + CI, system tester

- **W3:** fresh-install smoke test from the README **and** Appendix A, file every failure (Fri 16) → system test plan from Chaitanya's Lab02 sheet (Sun 18).

- **W4:** REAL/MOCK badge (Wed 21) → issues 7, 8b, 9 (Fri 23) → app in real mode, screen recording (Sun 25) → cycle 0 smoke test.
- **W5:** before/after view + back-end label in the app (Thu 29) → **test cycle 1, all system cases (Sat 31)**.
- **W6:** regression cycle 2 (Wed 4) → **demo video, two copies (Thu 5)** → **slide deck (Thu 5)** → report ch. 5 + 4d.
- **W7:** final slides; demo laptop fully offline-ready (local back-ups started once); Guide 4 checklist.

10 Open decisions for Chaitanya (before sharing)

1. **Base paper:** confirm which single paper the guide approved as "the base paper" (this document assumes Grounding DINO + SAM 2).
2. **Diwali and college dates:** confirm the festival days and the actual submission / viva date; shift Week 6–7 if needed.
3. **Report format:** does the department prescribe a black-book template? (T2.6 assumes we find out in Week 3.)
4. **Guide reviews:** if Prof. Dhore has scheduled reviews, put them right after Checkpoint 2 (25 Oct) or Checkpoint 3 (1 Nov), when we have real numbers to show.

Appendix A: Setting up model access (everyone)

Every member does Sections A.1–A.4 in Week 3, on their own laptop. A.5 (local models) is needed by Sumant and Chaitanya; others do it only if their laptop can. Budget about one hour. If you get stuck for more than 20 minutes, post the exact error in the group and tag Sumant.

> **Key rules (read before you create anything).**
> - **Use your own accounts.** Free limits are per account; one shared key would be used up by four people.
> - **Keys go only in `.env`.** Never in code, never in WhatsApp, never in a screenshot, never in a commit. `.env` is already in `.gitignore`. Check with `git status` that `.env` is never listed.
> - **Never add a credit card.** Everything here is free. If a page insists on a card, stop and tell the group.
> - If a key leaks, delete it in the console immediately and create a new one.

A.1 Project environment

Python 3.10 or 3.11. From the repository folder:

```
# Linux / macOS
python3.11 -m venv .venv && source .venv/bin/activate
python -m pip install -e '.[dev]'
cp .env.example .env
```

```
# Windows (PowerShell)
py -3.11 -m venv .venv ; .venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"
copy .env.example .env
```

Open `.env` in any text editor. The next sections tell you which lines to fill.

A.2 Groq (cloud reasoning: diagnosis, prescription text, image checks)

Free, no card. Limits for `qwen/qwen3.8-27b` on 12 Oct: 30 requests/min, 1,000/day, 8,000 tokens/min, 200,000 tokens/day; an image costs 2,048 tokens.

1. Go to `https://console.groq.com` and sign in with Google, GitHub or email.
2. Left menu → **API Keys** → **Create API Key**. Name it `agriagent-<yourname>`.
3. Copy the key now (it starts with `gsk_`). It is shown only once.

4. In `.env`: `GROQ_API_KEY=gsk_...` and leave `GROQ_MODEL_NAME=qwen/qwen3.8-27b`.
5. Your real limits: `https://console.groq.com/settings/limits`.

A.3 Cloudflare Workers AI (hosted Clef Flash triage)

Free allowance: 10,000 Neurons per day, resetting at 05:30 IST (00:00 UTC). Clef Flash uses about 3,455 Neurons per million input tokens, so the allowance is roughly 2.9 million tokens, or about 1,500 triage calls a day.

1. Go to `https://dash.cloudflare.com/sign-up`, sign up with email, verify the email.
2. Left menu → **AI** → **Workers AI** → **Use REST API**.
3. Copy the **Account ID** → in `.env`: `CLOUDFLARE_ACCOUNT_ID=...`
4. Click **Create a Workers AI API Token** → **Create API Token** → **Copy API Token** → in `.env`: `CLOUDFLARE_API_TOKEN=...` (A custom token needs the two permissions **Workers AI · Read** and **Workers AI · Edit**.)
5. Leave `CLOUDFLARE_MODEL=@cf/cloudflare/clef-flash`.

A.4 Check that everything answers

```
python scripts/check_providers.py           # all four checks
python scripts/check_providers.py groq      # just one: groq | cloudflare | local | clef-local
```

Expected for A.2 and A.3 (the two local checks FAIL until you do A.5, which is fine):

```
[PASS] groq          0.8s  qwen/qwen3.8-27b replied 'OK'; requests left today: 999
[PASS] cloudflare    1.1s  clef-flash answers: {'is_weed': {...}}
[FAIL] local         0.0s  Nothing is listening at http://127.0.0.1:8888/v1 ...
```

Post a screenshot of your PASS lines in the group. The script never prints keys, but check before posting anyway.

Error	Fix
`HTTP 401`	Key or token copied wrongly (extra space, missing characters). Create a new one.
`HTTP 429`	Rate limit. Wait a minute (Groq) or until 05:30 IST (Cloudflare).
`HTTP 404` / model not found (Groq)	The Preview model was withdrawn. Set `GROQ_MODEL_NAME=openai/gpt-oss-120b` and tell the group.
`not set in .env`	You edited `.env.example` instead of `.env`, or did not save. Run the script from the repository folder.

A.5 Local models with llama.cpp (offline backup)

Local models need no account and no internet once downloaded. They need disk space and, for speed, a GPU. `llama-server` gives the same kind of web API as Groq, on your own machine.

Install llama.cpp. It must be a build from **3 October 2026 or newer**; older builds cannot run Clef.

- Windows: download the latest release zip from `https://github.com/ggml-org/llama.cpp/releases`: `win-cuda` if you have an NVIDIA GPU, otherwise `win-vulkan` or `win-cpu`. Unzip; run commands from that folder.
- macOS: `brew install llama.cpp`.
- Linux: the release zip, or build from source:

```
git clone https://github.com/ggml-org/llama.cpp && cd llama.cpp
cmake -B build -DGGML_CUDA=ON && cmake --build build -j --target llama-server
```

Qwen3.5-4B with vision (about 3.5 GB download; runs on 4 GB+ of VRAM, or slowly on CPU):

```
llama-server -hf unsloth/Qwen3.5-4B-GGUF:Q4_K_M --port 8888 -c 8192 -ngl 99 -fa on --jinja
```

`-hf` downloads the model the first time, and should also fetch the vision file (`mmproj`). Check that the startup log mentions it. If it does not, download `mmproj-F16.gguf` from the same Hugging Face page and add `--mmproj mmproj-F16.gguf`. No GPU: use `-ngl 0`. Then: `python scripts/check_providers.py local`.

Clef Flash, local (6.5 GB download; text only, no images yet; needs 8 GB VRAM, or use `-ngl 20` to keep part of it in RAM):

```
llama-server -hf ggml-org/Clef-Flash-GGUF:Q4_K_M --port 8090 -c 4096 -b 4096 -ub 4096 -ngl 99 -fa on
```

A server running Clef answers only `/v1/systemone`, not chat. Then: `python scripts/check_providers.py clef-local`.

> **Which one to use when. Normal development and testing:** Groq + Cloudflare (fast, free, nothing to install). **Bulk image checks (T2.5, evaluation):** Cloudflare or local, because Groq's 8,000 tokens per minute allows only about three images a minute. **Demo day:** hosted first; both local servers started once beforehand so the models are already downloaded if the internet fails. **Never** run the local Clef server and the vision models on the 8 GB GPU at the same time.